



**Politechnika
Śląska**

Helm i Kubernetes

Instrukcja Laboratoryjna

Autor

Wojciech Krzyżanowski

Testowanie i weryfikacja

Michał Lach

Rok akademicki

25/26

Zadania – Wdrożenie DOCKER

W tym kroku weryfikujesz, że aplikacja działa bez Kubernetes.

Co się dzieje:

1. Budujesz obraz Dockera z aplikacji Flask
2. Uruchamiasz go lokalnie
3. Weryfikujesz domyślne zachowanie, gdy brak konfiguracji Kubernetes

Kroki do wykonania:

1. Sprawdź kod aplikacji:

Otwórz `api/app.py` i znajdź trasę `index`:

```
@flask_app.route("/")
def index():
    greeting = os.getenv(
        "APP_GREETING",
        "Hello from Flask running locally!"
    )
    return greeting, 200
```

Zwróć uwagę:

- Aplikacja odczytuje zmienną środowiskową `APP_GREETING`
- Używana jest wartość domyślna, gdy zmienna nie jest ustawiona

2. Zbuduj obraz kontenera:

```
make build IMAGE_NAME=kube-lab
```

3. Uruchom kontener lokalnie:

```
docker run --rm -p 5000:5000 kube-lab
```

4. Zweryfikuj aplikację: Otwórz przeglądarkę i wejdź na:

`http://localhost:5000`

Oczekiwany wynik w sekcji Greeting aplikacji:

Hello from Flask running locally!

5. Zatrzymaj uruchomiony kontener

Zadania – Wdrożenie HELM / KUBERNETES

W tym kroku wdrażasz ten sam obraz kontenera w Kubernetes przy użyciu Helm.

Co się dzieje:

1. Helm renderuje manifesty Kubernetes
2. Kubernetes wstrzykuje konfigurację przez ConfigMapy i Sekrety
3. Zachowanie aplikacji zmienia się bez przebudowy obrazu

Kroki do wykonania:

1. Upewnij się, że używasz poprawnego kontekstu Kubernetes:

```
make context KUBE_CONTEXT=docker-desktop
```

Jeśli Twój aktualny kontekst to już docker-desktop, ten krok nic nie zmieni.

2. Wdróż pełny stos:

```
make full
```

3. Sprawdź utworzone zasoby:

```
kubectl get pods -n kube-lab  
kubectl describe deployment kube-lab-lab -n kube-lab
```

4. Zweryfikuj aplikację przez Ingress:

Otwórz przeglądarkę i wejdź na:

<https://kube-lab-api.127.0.0.1.nip.io>

Oczekiwany wynik w sekcji Greeting aplikacji:

Hello from Flask running in Kubernetes via Ingress!

5. Na tym etapie zachęcamy do przeanalizowania Makefile znajdującego się w katalogu głównym projektu.

3. Zarządzanie konfiguracją i sekretami

Ta sekcja skupia się na obserwacji, jak konfiguracja przepływa z Helm do Kubernetes, a następnie do aplikacji.

ConfigMap: <https://kubernetes.io/docs/concepts/configuration/configmap/>

Sekrety: <https://kubernetes.io/docs/concepts/security/secrets/>

3.1 ConfigMap

Co się dzieje:

1. Helm generuje ConfigMapę na podstawie values.yaml
2. Kubernetes wstrzykuje wartości ConfigMapy jako zmienne środowiskowe
3. Aplikacja odczytuje je w czasie działania

Kroki do wykonania:

1. Sprawdź kod aplikacji:

```
@flask_app.route("/")
def index():
    return render_template(
        "index.html",
        greeting=os.getenv(
            "APP_GREETING",
            "Hello from Flask running locally!"
        ),
        app_name=os.getenv("APP_NAME"),
        has_secret=os.getenv("API_KEY") is not None
    ), 200
```

Zwróć uwagę:

- APP_GREETING steruje tekstem powitania
 - Istnieje wartość domyślna, gdy zmiennej brakuje
2. Sprawdź wartości Helm:

Otwórz devops/kube-lab/values.yaml i znajdź:

```
config:
  APP_GREETING: "Hello from Flask running in Kubernetes via Ingre
ss!"
```

3. Sprawdź wyrenderowaną ConfigMapę:

```
kubectl get configmap kube-lab-lab -n kube-lab -o yaml
```

4. Zweryfikuj zachowanie w czasie działania:

Odśwież stronę aplikacji w przeglądarce i obserwuj powitanie.

3.2 Sekrety

Sekrety podążają podobnym przepływem jak ConfigMapy, ale z bardziej restrykcyjnym podejściem.

Co się dzieje

4. Sekrety nie są przechowywane w values.yaml
5. Sekrety pochodzą z lokalnego pliku .env
6. Sekrety Kubernetes są tworzone imperatywnie
7. Helm jedynie odwołuje się do sekretu po nazwie

Kroki

1. Sprawdź logikę aplikacji:

```
has_secret = os.getenv("API_KEY") is not None
```

Zwróć uwagę:

- Aplikacja sprawdza jedynie obecność
- Wartość sekretu nigdy nie jest ujawniana

2. Sprawdź wartości Helm:

Otwórz devops/kube-lab/values.yaml i znajdź:

```
api:
  secret:
    name: api-secret
```

Zwróć uwagę:

- Helm wie, którego sekretu użyć
- Helm nie zna wartości sekretu

3. Sprawdź, jak Deployment odwołuje się do sekretu:

Otwórz devops/kube-lab/templates/deployment.yaml i znajdź:

```
envFrom:
- configMapRef:
    name: kube-lab-lab
- secretRef:
    name: api-secret
```

4. Sprawdź sekret w Kubernetes:

```
kubectl get secret api-secret -n kube-lab
kubectl get secret api-secret -n kube-lab -o yaml
```

5. Zweryfikuj zachowanie w czasie działania:

Odwiedź panel aplikacji i obserwuj sekcję Secrets.

Zadania HELM TEMPLATES AND VALUES

Wszystkie zmiany w tej sekcji muszą być wykonane w **wartościach lub szablonach Helm**, o ile wyraźnie nie zaznaczono inaczej.

Rozgrzewka: Podstawy wartości Helm

Te zadania skupiają się na zapoznaniu się z wartościami Helm i tym, jak wpływają na działający Deployment.

- 1) Ustaw liczbę replik API na 3
- 2) Ustaw politykę pobierania obrazu na Always
- 3) Zmień wartość konfiguracji powitania (APP_GREETING) na „Welcome to the Lab”
Uruchom aplikację ponownie i zweryfikuj, czy powitanie zmieniło się w aplikacji.
- 4) Dodaj flagę środowiskową aplikacji (APP_ENV) w ConfigMap o wartości lab
- 5) Zmień wartość API_KEY.

Dostrajanie wdrożenia

Te zadania koncentrują się na konfiguracji wdrożenia w stylu produkcyjnym.

- 6) Skonfiguruj żądania zasobów
 - cpu: 100m
 - memory: 128Mi
- 7) Skonfiguruj limity zasobów
 - cpu: 250m
 - memory: 256Mi
- 8) Dodaj adnotację poda
 - Klucz: example.com/release
 - Wartość: lab
- 9) Dodaj adnotacje Service dla scrapingu Prometheus
 - prometheus.io/scrape: “true”
 - prometheus.io/port: “5000”
- 10) Skonfiguruj strategię aktualizacji kroczącej

- maxSurge: 1
- maxUnavailable: 0
- Lokalizacja: api.strategy

Rozmieszczenie i routing

Te zadania kontrolują, gdzie działają pody i jak dociera do nich ruch.

- 11) Dodaj selektor węzłów
 - kubernetes.io/os: linux
- 12) Dodaj tolerację
 - key: workload
 - operator: Exists
 - effect: NoSchedule
- 13) Dodaj antyafinity poda
 - Rozprosz pody według app.kubernetes.io/name
 - Preferowana lub wymagana affinity jest akceptowalna
- 14) Dodaj dodatkowy host Ingress
 - Host: kube-lab.local
- 15) Włącz TLS dla nowego hosta
 - Nazwa sekretu: kube-lab-tls
 - Musi obejmować kube-lab.local

Autoskalowanie

Te zadania włączają i konfigurują Horizontal Pod Autoscaler.

- 16) Włącz autoskalowanie
 - autoscaling.enabled: true
- 17) Ustaw minimalną liczbę replik
 - autoscaling.minReplicas: 2
- 18) Ustaw maksymalną liczbę replik
 - autoscaling.maxReplicas: 5
- 19) Skonfiguruj docelowe wykorzystanie CPU
 - autoscaling.targetCPUUtilizationPercentage: 60

Nadpisanie

To zadanie wprowadza nadpisanie zależne od środowiska.

- 20) Utwórz plik nadpisanie dla środowiska deweloperskiego
 - Plik: devops/kube-lab/values.dev.yaml
 Plik musi zawierać:

- api.replicas: 1
- api.ingress.hosts: ["kube-lab-dev.127.0.0.1.nip.io"]

Ocena

1. Uruchom automatyczny grader:

```
make grade
```

- 1) Sprawdź wynik

- Każde zadanie jest oznaczone jako pass lub fail
- Wyświetlany jest wynik końcowy na 20 punktów